Weekly Documentation Assignment By: Fredricka David

1. Editing Shell Scripts

I learned how to create and edit shell scripts using the `nano` text editor. Shell scripts allow me to run multiple commands in sequence and automate tasks in Linux.

I practised moving around the editor, editing text, saving files and exiting the editor. I also learned how to cut and paste lines without typing them again.

Commands and shortcuts used

Command	Explanation
`nano test.sh`	Opens or creates a file named `test.sh` in the Nano editor.
`Ctrl + O`	Saves the file without closing the editor.
`Ctrl + X`	Exits the editor.
`Ctrl + K`	Cuts the current line.
`Ctrl + U`	Pastes previously cut text.
`Ctrl + W`	Searches for text in the file.
`Ctrl + G`	Displays the help menu.
`Ctrl + Y` / `Ctrl + V`	Moves one page up or down.

A simple shell script can contain commands such as `echo` to display messages and `date` to display the current date and time.

2. Variables in Shell Scripts

I learned that variables store information that can be reused in a script. For example:

```
#!/bin/bash
ANIMAL="penguin"
echo "My favorite animal is a $ANIMAL"
```

The output is:

```
My favorite animal is a penguin
```

When assigning a value to a variable, there must be no spaces around the equals sign. The dollar sign (`$`) is used to access the value stored in a variable.

I also learned that a variable can store the output of a command, accept input from a user or receive arguments passed to a script.

Important commands and variables

Command or variable	Explanation
`pwd`	Displays the current working directory.
`CURRENT_DIRECTORY=$(pwd)`	Stores the output of `pwd` in a variable.
`read NAME`	Accepts input from the user and stores it in `NAME`.
`$1`	Represents the first argument passed to a script.
`$0`	Represents the script's name.
`$?`	Displays the exit status of the previous command.
`exit 1`	Ends a script with a non-zero status, commonly indicating an error.

An exit status of `0` normally means a command completed successfully, while a non-zero status indicates that something went wrong or that a particular condition was not met.

3. Conditional Statements

I learned how to use conditional statements to make scripts perform different actions depending on whether a condition is true or false.

The `if`, `elif` and `else` statements allow a script to check conditions and choose what to do next. The `fi` keyword closes an `if` statement.

For example:

```
#!/bin/bash

if [ "$1" = "hello" ]; then
    echo "hello yourself"
elif [ "$1" = "goodbye" ]; then
    echo "nice to have met you"
else
    echo "I didn't understand that"
fi
```

This script checks the first argument provided when running it and displays a suitable message.

I also learned about the `test` command and square brackets (`[ ]`), which can be used to check whether files exist, compare numbers and compare strings. The `case` statement provides another way to handle multiple possible values.

These concepts are useful when writing scripts that need to make decisions, check files or respond to user input.

4. Random Access Memory (RAM) and Swap Space

I learned that RAM is the physical memory a computer uses to store data and instructions that programs need while running. When physical memory becomes limited, Linux can use swap space on a storage device to hold less recently used data temporarily.

Swap space helps the system manage memory, but accessing data from disk is generally slower than accessing RAM.

Command used

```
free -m
```

This command displays memory usage in megabytes, including total, used and available memory, as well as swap space.

Another useful option is:

```
free -g
```

This displays memory information in gigabytes.

I learned that monitoring memory usage helps identify whether a system is running low on memory and may be experiencing performance problems.

5. Linux Processes and the `/proc` Directory

I learned that a process is a running program. Linux assigns each process a Process ID (PID), which helps identify and manage it.

The `/proc` directory is a virtual filesystem that provides information about running processes, memory usage, hardware and the Linux kernel. Its contents are generated by the system rather than stored as ordinary files on disk.

Commands used

Command	Explanation
`ls /proc`	Lists process information and other system information available through `/proc`.
`cat /proc/meminfo`	Displays detailed memory information.
`cat /proc/cmdline`	Displays the parameters passed to the kernel at startup.
`cat /proc/modules`	Lists kernel modules currently loaded.
`cat /proc/loadavg`	Displays system load averages and process-related information.
`pstree`	Displays running processes in a tree structure.

`ps`	Shows a snapshot of running processes.
`top`	Monitors processes and system resource usage in real time.

I also learned about parent and child processes. A parent process starts another process, which becomes its child. The PID identifies a process, while the PPID identifies its parent process.

The first process started during system initialization has PID 1. Other processes may be started by it or by other running processes.

6. Monitoring Processes and System Performance

I learned how to use `top` to monitor running processes, CPU usage, memory consumption and system load. Unlike `ps`, which provides a snapshot, `top` updates its information continuously.

The load average shows the system's average workload over the previous one, five and fifteen minutes. It must be considered alongside the number of CPU cores to understand how busy a system is.

Additional commands

```
top
```

Displays a live view of running processes and resource usage. Press `q` to exit.

```
free -s 10
```

Updates memory statistics every ten seconds.

While using `top`, the `K` key can be used to request termination of a process, and the `R` key can be used to change its priority. These actions require care because terminating or reprioritising the wrong process can affect system performance.

I learned that regularly monitoring CPU and memory usage can help identify resource-intensive processes before they cause serious problems.

7. Kernel Messages and Troubleshooting

I learned that kernel messages contain information about hardware, drivers and system-level events. These messages can help diagnose problems during startup or while the system is running.

Command used

```
dmesg | grep -i usb
```

- `dmesg` displays kernel messages.
- `|` passes the output of one command to another.
- `grep -i usb` searches for lines containing `usb`, ignoring differences between uppercase and lowercase letters.

This is useful when investigating hardware-related problems, such as USB devices that are not working correctly.

I also learned that kernel settings can be exposed through `/proc/sys`. Changing some of these settings can affect system behaviour, so such changes should only be made when necessary and with an understanding of their impact.

8. Networking Fundamentals

I learned about important networking concepts used when computers communicate with one another.

- **Network packets:** Small units of data sent between devices.
- **IP addresses:** Numerical addresses used to identify devices on a network.
- **Subnet masks:** Define which IP addresses belong to a particular network.
- **Hostnames:** Human-readable names used to identify network devices.
- **DHCP:** Automatically assigns IP addresses and other network settings to devices.
- **DNS:** Translates domain names into IP addresses.
- **Ethernet:** A common technology for wired network connections.
- **TCP/IP:** A collection of protocols that enables communication between devices.

I also learned the difference between IPv4 and IPv6. IPv4 uses 32-bit addresses, while IPv6 uses 128-bit addresses and provides a much larger address space.

9. Network Configuration and Name Resolution

I learned that Linux uses configuration files to determine how hostnames are resolved into IP addresses.

File	Purpose
`/etc/hosts`	Contains local mappings between hostnames and IP addresses.
`/etc/resolv.conf`	Contains DNS name server settings.
`/etc/nsswitch.conf`	Defines the order in which sources are checked for hostname information.

For example:

```
cat /etc/hosts
cat /etc/resolv.conf
cat /etc/nsswitch.conf
```

The `cat` command displays the contents of each file.

I learned that the `hosts` entry in `/etc/nsswitch.conf` can specify whether local files or DNS should be checked first. Understanding this process is useful when troubleshooting problems where a hostname cannot be resolved.

I also learned that network interface configuration files and service commands vary between Linux distributions. For this reason, it is important to use the correct configuration method for the system being managed.

10. Routing and Network Connections

I learned that routing determines where network packets should be sent to reach their destination. A default gateway is used when a destination does not match a more specific route.

Commands used

Command	Explanation
`route`	Displays the network routing table on systems that support it.
`route -n`	Displays routing information using numerical addresses.
`ip route show`	Displays the routing table using the modern `ip` command.
`netstat -i`	Displays network interface statistics.
`netstat -r`	Displays the routing table.
`netstat -tln`	Lists listening TCP ports using numerical addresses.

I learned that `netstat` is considered obsolete on many modern Linux distributions, with tools such as `ss` and `ip` providing the recommended alternatives.

For example:

```
ip route show
ss -tln
ip -s link
```

These commands help inspect routing information, listening TCP ports and network interface statistics.

A listening port indicates that a service is waiting for connections on that port. However, whether the service can be reached from another machine also depends on the network configuration and firewall rules.

11. Connecting to Remote Machines Using SSH

I learned that Secure Shell (SSH) allows users to connect securely to remote Linux machines and execute commands.

The general syntax is:

```
ssh username@hostname
```

The username identifies the account to use, while the hostname or IP address identifies the remote machine.

After connecting, I can run commands on the remote system. The `exit` command closes the SSH session and returns me to the local terminal.

I also learned about SSH host key fingerprints. When connecting to a host for the first time, SSH may ask me to verify its identity. The host key is stored locally so it can be checked during future connections.

If SSH reports that a remote host's identification has changed, I should investigate and verify the change before proceeding. The warning may have a legitimate explanation, such as a server being reinstalled, but it can also indicate a security risk.

Key Takeaways

This learning section helped me understand how Linux shell scripts work and how they can be used to automate tasks. I learned how to edit scripts, use variables, accept user input, check command results and make decisions using conditional statements.

I also developed a better understanding of Linux system administration, including memory management, running processes, kernel messages and system monitoring. The networking topics helped me understand IP addressing, DNS, routing, listening ports and remote access through SSH.

These concepts are important for cloud engineering and system administration because they help with automation, troubleshooting, monitoring and managing Linux servers.

Screenshots of Hands-on Work

I will include screenshots of the practical work completed during the lessons to demonstrate my understanding. Useful screenshots include:

1. Creating and editing a shell script using `nano`.
2. Running a script that uses variables and displays output.
3. Using conditional statements to check different conditions.
4. Checking memory usage with `free -m`.
5. Viewing running processes using `ps`, `pstree` or `top`.

6. Filtering kernel messages using `dmesg` and `grep`.
7. Viewing the routing table and listening ports.
8. Connecting to a remote machine using SSH, where available.

